

Qhaphela — Job Posting Fraud Detector

Project documentation: waterfall development model and fraud-scoring algorithm pseudocode

UNESCO Youth Hackathon 2026 — AI and Media & Information Literacy track

1. Waterfall Development Model

Qhaphela was built following a sequential waterfall process, with each phase completed and validated before the next began. The stages below reflect the actual build order used for the project.

1

Requirements Analysis

- Research proposal: "A Machine Learning Approach for Detecting Fraudulent Job Postings to Fight Identity Theft Among South African Job Seekers"
- SA-specific fraud patterns identified: fake POPIA clauses, unverifiable B-BBEE claims, WhatsApp migration, upfront-payment requests, ID/banking requests before interview, salary-mismatch lures
- UNESCO Youth Hackathon brief: AI and MIL track, deadline 16 August 2026

2

System Design

- Two-layer feature architecture: interpretable rule-layer signals (regex-matched SA fraud patterns) combined with TF-IDF lexical features
- Model choice: Random Forest over deep/transformer models, favouring interpretability (SHAP-compatible) over raw accuracy, per the proposal's opacity argument
- Product surfaces designed: Chrome extension (primary) with an in-page panel, plus a secondary web-app demo surface backed by the same scoring API

3

Implementation

- Backend: FastAPI scoring service (`app.py`) serving a trained scikit-learn Random Forest + TF-IDF vectorizer + SHAP TreeExplainer
- Frontend: Manifest V3 Chrome extension (content script, background service worker, toolbar popup) plus a React/Express web demo
- Both frontend surfaces call one shared `/score` API, so results are always consistent

4

Testing & Verification

- Unit-level checks against known scam and legitimate sample postings
- Live testing against real South African job listings on Indeed surfaced a generalisation gap: the classifier over-weighted generic recruiting vocabulary from synthetic training data
- Tactical mitigation shipped: score is capped unless at least one interpretable rule-based signal is actually present in the text

5

Deployment & Maintenance

- Extension loaded and run locally against the FastAPI model service; source pushed to a shared GitHub repository
- Ongoing maintenance: broadened job-platform coverage, fixed a dead reference link, and revised the in-page panel layout and readability based on direct testing feedback
- Open item: full model retraining on improved data, planned as follow-up work before further evaluation

2. Fraud-Scoring Algorithm — Pseudocode

The core scoring routine served by the FastAPI /score endpoint, called by both the Chrome extension and the web app.

```
FUNCTION score_posting(text):  
  
    // 1. Feature extraction  
    tfidf_vector ← TFIDF_VECTORIZER.transform(text)  
    rule_vector ← EXTRACT_RULE_FEATURES(text)  
        // popia_clause_with_doc_request, bbbee_claim_no_cert,  
        // whatsapp_migration, upfront_payment_request,  
        // id_or_banking_request, urgency_language,  
        // salary_mismatch_ratio, freemail_contact,  
        // posting_length_norm  
    feature_vector ← COMBINE(tfidf_vector, rule_vector)  
  
    // 2. Model inference  
    probability ← RANDOM_FOREST.predict_proba(feature_vector)  
    score ← ROUND(probability × 100)  
  
    // 3. Safety-net floor: explicit evidence overrides a low model score  
    floors ← HARD_FLOOR_FLAGS(text)  
        // requests ID/banking details, or an upfront payment  
    IF floors is not empty THEN  
        score ← MAX(score, 75)
```

```

END IF

// 4. Tactical ceiling: don't trust an evidence-free model score
IF NO rule_vector signal fired THEN
    score ← MIN(score, 25)
END IF

// 5. Tier classification
IF score < 30 THEN tier ← LOW
ELSE IF score ≤ 70 THEN tier ← MEDIUM
ELSE tier ← HIGH
END IF

// 6. Explainability
reasons ← SHAP_EXPLAIN(RANDOM_FOREST, feature_vector)
highlights ← MATCH_PHRASES(text)
    // literal red-flag substrings, for in-page underlining

RETURN { score, tier, reasons, highlights, floors }

END FUNCTION

```

Reflects the current implementation in qhaphela/app.py and qhaphela/features.py, including the tactical score ceiling added after live testing against real job postings (Step 4).